先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 15 – Deep Learning I: Neural Networks and Backpropagation

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Three periods: represent, predict, and learn

Time	Main question	What we will calculate
45 min	Why do hidden layers need nonlinearity?	XOR, activations, shapes, parameters
45 min	How does a network make a prediction?	A complete forward pass and suitable losses
45 min	How does a prediction error change weights?	Numerical backpropagation, updates, initialization

One continuous example

A two-input, two-hidden-unit network will connect the architecture, forward values, loss, and gradients.

By the end

Explain each symbol, reproduce one update, and say what an added hidden layer changes.

A neural network extends models we already know

$$\begin{aligned}
\text{Linear regression:} &\quad \hat{y} = w^\top x + b, \\
\text{Logistic regression:} &\quad p(y = 1 \mid x) = \sigma(w^\top x + b), \\
\text{Neural network:} &\quad h = \phi(W_1 x + b_1), \quad p = \sigma(w_2^\top h + b_2).
\end{aligned}$$

- The output model still has the same statistical meaning.
- The hidden layer replaces fixed input features by learned features h.
- Labels influence both the output weights and the feature-producing weights.

Connection

Training a network jointly learns a representation and a predictor.

Fixed features and learned features solve the same need

$$\text{fixed: } q(x) = (x_1, x_2, x_1 x_2), \qquad \text{learned: } h(x) = \phi(Wx + b).$$

- Both transform the input before a simple output model makes a prediction.
- A fixed interaction is specified by the designer; a hidden feature changes during training.
- The representation must help distinguish the target, not merely increase dimension.

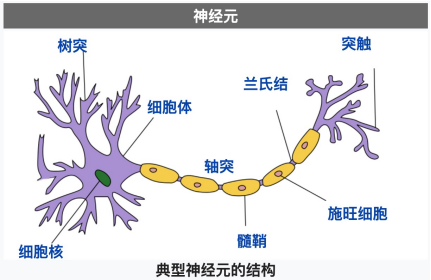
Binary inputs

The interaction $x_1 x_2$ is 1 only at $(1, 1)$. Together with $x_1 + x_2$, it gives XOR as $x_1 + x_2 - 2x_1 x_2$.

Meaning

A useful representation can make a difficult decision simple; learning the representation is an additional modeling choice.

Biology motivates the idea; mathematics defines the unit



典型神经元的结构

Engineering abstraction

- Inputs: measured features or activations from the previous layer.
- Weights: adjustable strengths of the connections.
- Bias: an adjustable offset.
- Activation: a nonlinear response to the weighted sum.

Interpret the model carefully

A feed-forward unit is a useful computation, not a simulation of all biological processes.

One unit: weighted sum, bias, then activation

$$z = w^\top x + b, \qquad a = \phi(z).$$

Quantity	Example	Meaning
x	$(2, -1)^\top$	observed input
w, b	$(0.5, 1)^\top,\ 0.2$	learned parameters
z	$0.5(2) + 1(-1) + 0.2 = 0.2$	score before activation
a	$\mathrm{ReLU}(0.2) = 0.2$	value sent to the next layer

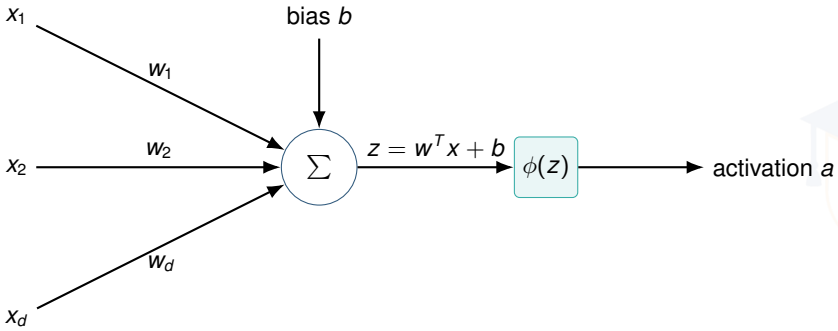
Bias matters

Changing b shifts the boundary without changing its normal direction w.

Try another input

$x = (0, 1)^\top$ gives $z = 1.2$; the same parameters process every example.

One unit: weighted sum, bias, then activation



Weights and bias change different aspects of a boundary

For $z = 2x_1 - x_2 + b$ and prediction 1 when $z > 0$:

$$b = 0 : \ x_2 = 2x_1, \qquad b = 1 : \ x_2 = 2x_1 + 1.$$

- Increasing b translates the boundary while retaining its slope.
- Changing the ratio of the weights changes its orientation.
- The activation transforms the score, but the score still comes from these contributions.

At $x = (1, 2)$

The scores are 0 and 1 for the two biases. The second setting predicts class 1 under the stated rule.

Connection

Biases are learned parameters, not an extra measured feature or a different bias for each sample.

XOR cannot be separated by one affine threshold

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

For prediction 1 when $w_1 x_1 + w_2 x_2 + b > 0$:

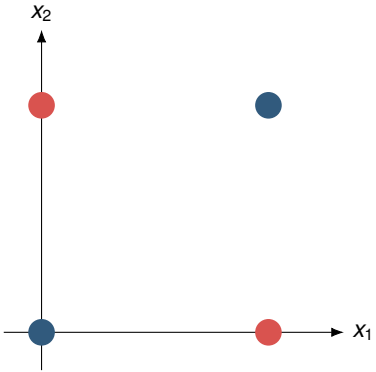
$$b \le 0, \quad w_1 + b > 0, \quad w_2 + b > 0,$$
$$w_1 + w_2 + b \le 0.$$

Adding the two positive inequalities gives $w_1 + w_2 + 2b > 0$, so $w_1 + w_2 + b > -b \ge 0$: a contradiction.

What failed?

The restriction to one straight boundary failed; collecting more copies of the four points cannot remove it.

XOR cannot be separated by one affine threshold



Opposite corners share a label. No single affine boundary separates them.

The XOR geometry also gives a midpoint contradiction

The positive points $(1,0),(0,1)$ and the negative points $(0,0),(1,1)$ have the same midpoint $(0.5,0.5)$.

$$z\big((a+b)/2\big) = \big(z(a)+z(b)\big)/2 \quad \text{for affine } z.$$

- Both positive scores exceed zero, so their midpoint score must exceed zero.
- Both negative scores are at most zero, so their midpoint score must be at most zero.
- One point cannot satisfy both conditions.

Meaning

The arrangement of labels causes the limit. Training longer or replicating examples does not alter that arrangement.

AND and OR each need only one threshold

Use $H(t) = 1$ for $t > 0$ and $H(t) = 0$ otherwise.

$$\text{OR} = H(x_1 + x_2 - 0.5), \qquad \text{AND} = H(x_1 + x_2 - 1.5).$$

Input	Sum	OR score	AND score
$(0, 0)$	0	-0.5	-1.5
$(0, 1)$ or $(1, 0)$	1	0.5	-0.5
$(1, 1)$	2	1.5	0.5

Read the thresholds

OR accepts a sum of at least one; AND accepts only the sum two.

Bridge to the hidden layer

Combining these two easy questions yields the harder XOR decision; hidden units need not individually solve the whole task.

Two hidden tests make XOR separable

Let $H(t) = 1$ if $t > 0$, and 0 otherwise.

$$h_1 = H(x_1 + x_2 - 0.5), \quad h_2 = H(x_1 + x_2 - 1.5), \quad \hat{y} = H(h_1 - h_2 - 0.5).$$

(x_1, x_2)	h_1 (OR)	h_2 (AND)	$h_1 - h_2$	$\hat{y}$
$(0, 0)$	0	0	0	0
$(0, 1)$	1	0	1	1
$(1, 0)$	1	0	1	1
$(1, 1)$	1	1	0	0

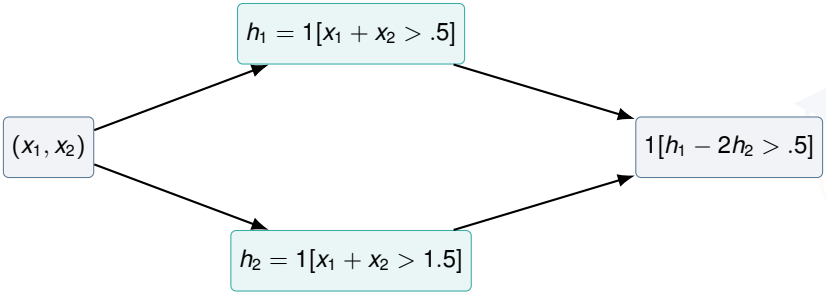
New coordinates

The output separates $(1, 0)$ from $(0, 0)$ and $(1, 1)$ in hidden-feature space.

Training issue

A hard threshold illustrates representation; its derivative is unsuitable for ordinary backpropagation.

Two hidden tests make XOR separable



The same output line works in the hidden coordinates

Using hidden coordinates $(h_1, h_2) = (\mathrm{OR}, \mathrm{AND})$:

$$\text{positive: } (1,0), \qquad \text{negative: } (0,0), (1,1).$$

$$z_{\mathrm{out}} = h_1 - h_2 - 0.5.$$

- The positive hidden point gets score 0.5.
- Both negative hidden points get score -0.5.
- The original four inputs map to three hidden points; merging two same-label inputs causes no problem here.

Meaning

Separability is a property of the representation as well as the labels. A hidden layer can change that representation before the output classifier acts.

A ReLU hidden layer can also represent XOR

For binary inputs, define

$$r = x_1 + x_2, \qquad h_1 = \mathrm{ReLU}(r), \quad h_2 = \mathrm{ReLU}(r-1), \quad f(x) = h_1 - 2h_2.$$

Input sum r	h_1	h_2	$f(x)$	XOR target
0	0	0	0	0
1	1	0	1	1
2	2	1	0	0

- Each hidden unit is simple; combining their outputs bends the response.
- This is an exact construction on the four binary inputs, not a probability model yet.
- ReLU is differentiable away from zero; software chooses a subgradient at zero.

ReLU pieces can build a localized response

$$f(t) = \mathrm{ReLU}(t) - 2\,\mathrm{ReLU}(t-1) + \mathrm{ReLU}(t-2).$$

t	Three hidden values	$f(t)$	Meaning
0.5	$(0.5, 0, 0)$	0.5	rising segment
1.5	$(1.5, 0.5, 0)$	0.5	falling segment
3	$(3, 2, 1)$	0	cancellation beyond two

- Each hidden response is simple; their weighted sum creates a different shape.
- The shifted biases place slope changes at $0, 1, 2$.

Meaning

Nonlinear features can cooperate to respond in a specific input region, even though each individual ReLU is not localized.

An expressible solution is different from a learned solution

- A constructed XOR network shows that some parameters solve the four-point problem.
- Training begins from other parameters and follows the chosen loss gradients.
- Nonconvex objectives can have different optimization paths; data order and initialization can matter.
- Good training fit must still be distinguished from performance on unseen inputs.

Concrete contrast

A manually designed threshold network has the correct truth table but almost no useful threshold derivative. A differentiable network can be trained, but still needs an effective optimization process.

Three questions

Can the architecture express the target? Can training find useful parameters? Does the result generalize?

Hidden features are intermediate questions

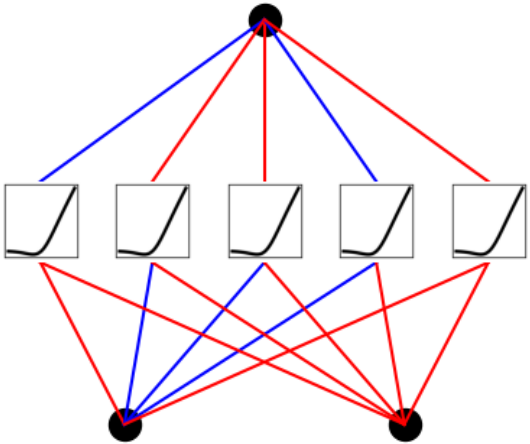
Input task	Possible hidden information	Output decision
XOR	at least one input; both inputs	exactly one input
Digit recognition	strokes, corners, combinations	digit class
Tabular prediction	nonlinear interactions of measurements	risk or numeric target

- Hidden coordinates need not have a unique human name.
- Different parameter settings can represent the same predictor.
- Useful features depend on the loss and training examples.

Bridge to feature engineering

A network learns combinations during training; domain knowledge still helps choose inputs and evaluate errors.

Hidden features are intermediate questions



Hidden units can be permuted without changing the prediction

For $f(x) = v_1 h_1(x) + v_2 h_2(x) + c$, swap the two hidden units and their outgoing weights:

$$f(x) = v_2 h_2(x) + v_1 h_1(x) + c.$$

- The function is unchanged, although parameter indices differ.
- Hidden unit number one has no permanent semantic identity.
- Features can work together; an individual activation is not a complete explanation of a prediction.

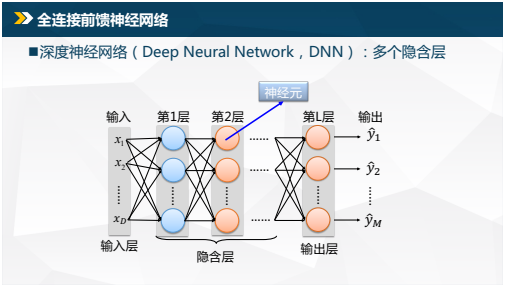
Numbers

$h = (0.3, 0.7)$ and $v = (2, -1)$ give -0.1. Swapping both pairs still gives -0.1; swapping only the features generally does not.

Meaning

Interpret a hidden value together with its connections and the task, rather than assigning a name from its index alone.

Read a network from left to right



Read the arrows

- Input layer: stores x.
- Hidden layers: compute new activations.
- Output layer: produces task-specific scores.
- Each connection has a weight; each non-input unit has a bias.

Depth

Count learned transformations; an input node does not perform a learned transformation.

Shapes say which computation is possible

For one column-vector example with $d = 4$ inputs, $m = 3$ hidden units, and $K = 2$ outputs:

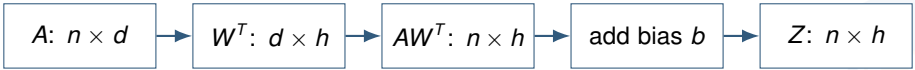
$$z_1 = W_1 x + b_1, \quad h = \phi(z_1), \quad z_2 = W_2 h + b_2.$$

Object	Shape	Why
x	4×1	four input features
W_1, b_1	$3 \times 4,\ 3 \times 1$	one row for each hidden unit
z_1, h	3×1	three hidden values
W_2, b_2	$2 \times 3,\ 2 \times 1$	two output scores
z_2	2×1	one score per output

Rule

The inner dimensions must agree; the bias must match the output dimension.

Shapes say which computation is possible



Rows of a dense weight matrix are individual units

$$W = \begin{bmatrix} 1 & 0 & -1 \\ 0.5 & 0.5 & 0 \end{bmatrix}, \quad x = \begin{bmatrix} 2 \\ 4 \\ 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

$$z = Wx + b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad \mathrm{ReLU}(z) = (1, 2)^\top.$$

- Row one uses the difference $x_1 - x_3$.
- Row two uses the average of x_1, x_2, then subtracts one.
- The input has three coordinates; the layer creates two different features.

Meaning

The matrix form performs the familiar weighted-sum unit several times at once. Each row has its own bias and nonlinear response.

Two examples reuse one bias vector

For row-wise samples:

$$X = \begin{bmatrix} 2 & 4 & 1 \\ 1 & 2 & 3 \end{bmatrix}, \quad Z = XW^\top + \mathbf{1}b^\top = \begin{bmatrix} 1 & 2 \\ -2 & 0.5 \end{bmatrix}.$$

- Both rows use the previous slide's W and $b = (0, -1)$.
- The output is 2×2: two examples, two units.
- The first output row agrees with the single-example calculation.

After ReLU

$H = \begin{bmatrix} 1 & 2 \\ 0 & 0.5 \end{bmatrix}$. Different examples activate the same unit differently.

Meaning

Broadcasting repeats the addition, not the number of learned bias parameters.

Parameter count is independent of batch size

For a dense layer with d inputs and m outputs:

$$\#\text{parameters} = dm + m.$$

Layer	Weights	Biases	Total
$4 \to 3$	12	3	15
$3 \to 2$	6	2	8
Whole network	18	5	23

- Processing 100 examples reuses these 23 parameters.
- A wider hidden layer increases representation capacity and memory use.
- More parameters do not guarantee better validation performance.

Calculate

$784 \to 128 \to 10$ has $(784 + 1)128 + (128 + 1)10 = 101{,}770$ parameters.

Widening a layer has a calculable cost

For $d = 20$ inputs and $K = 3$ outputs:

$$P(m) = (20 + 1)m + (m + 1)3 = 24m + 3.$$

Hidden width m	Total parameters	Change
8	195	starting network
16	387	192 added parameters
32	771	384 more than width 16

- Every new hidden unit adds incoming weights, one bias, and outgoing weights.
- Capacity increases, but extra parameters need useful training information.

Meaning

Choose width to fit the task and data; an exact count helps compare alternatives before training.

Activations change both values and derivatives

Activation	Value $\phi(z)$	Derivative away from kinks	Typical use
Sigmoid	$1/(1+e^{-z})$	$\phi(z)(1-\phi(z))$	binary output, gates
Tanh	$\tanh z$	$1-\tanh^2 z$	bounded hidden state
ReLU	$\max(0,z)$	0 or 1	common hidden layer
Leaky ReLU	$\max(\alpha z, z)$	α or 1	negative-side gradient

For Leaky ReLU here, assume $0 < \alpha < 1$.

- A hidden activation determines the nonlinear representation.
- Its derivative determines how local changes influence earlier layers.
- The output activation must match the meaning of the prediction.

Avoid a blanket claim

ReLU helps preserve derivatives on its positive side; it does not guarantee stable gradients in every deep network.

Compare activation values and slopes at the same input

At $z = 2$:

Activation	Value	Local derivative
Sigmoid	0.8808	0.1050
Tanh	0.9640	0.0707
ReLU	2	1

- Bounded activations compress a large positive score.
- A smaller derivative attenuates a local change passing through that activation.
- ReLU preserves this positive-side slope, but weights still affect the whole-network derivative.

Small change

For $\Delta z = 0.01$, predicted activation changes are about 0.00105, 0.000707, and 0.01 respectively.

Sigmoid saturates: the derivative explains why

$$a = \sigma(z), \qquad \frac{da}{dz} = a(1-a).$$

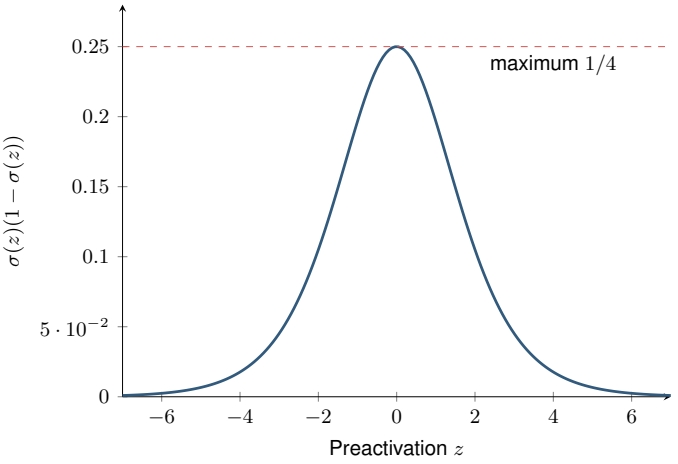
z	a	$a(1-a)$	Response to a small change in z
0	0.500	0.250	largest local response
2	0.881	0.105	smaller response
5	0.993	0.00665	nearly saturated

- Far from zero, changing the score barely changes the activation.
- Repeated small derivatives can weaken the gradient through many layers.
- Weight matrices also matter; the activation derivative alone is not the whole network gradient.

Magnitude intuition

$0.25^5 \approx 0.00098$: five such scalar factors already shrink a signal substantially.

Sigmoid saturates: the derivative explains why



Tanh uses a doubled sigmoid argument

$$\tanh z = 2\sigma(2z) - 1, \qquad \frac{d}{dz}\tanh z = 1 - \tanh^2 z.$$

- The factor two belongs inside sigmoid as well as outside it.
- At $z = 1$, $2\sigma(2) - 1 \approx 0.7616$, while $2\sigma(1) - 1 \approx 0.4621$ is different.
- Tanh is centered around zero but still saturates for large magnitudes.

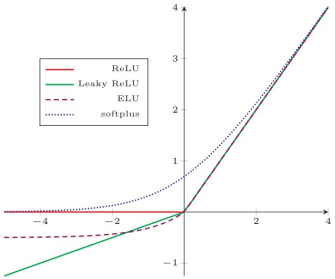
Derivative check

At $z = 0$, the derivative is 1; at $z = 3$, it is about 0.00987.

Meaning

A zero-centered output can be useful, but saturation and the chosen task still determine whether the activation is appropriate.

ReLU variants keep different negative-side responses



Read value and slope together

- ReLU: negative output and gradient are zero.
- Leaky ReLU: negative slope $0 < \alpha < 1$.
- ELU: smooth negative saturation.
- Softplus: smooth approximation to ReLU.

For $z = -2$ and $\alpha = 0.1$: ReLU gives 0; Leaky ReLU gives -0.2.

A practical distinction

A unit inactive for every training example receives no ReLU gradient through that branch.

Leaky ReLU sends a smaller gradient through a negative input

$$a = \max(\alpha z, z), \qquad \alpha = 0.1, \quad z = -2.$$

Then $a = -0.2$, and for incoming gradient $\partial L/\partial a = 3$:

$$\frac{\partial L}{\partial z} = 3\alpha = 0.3.$$

- Ordinary ReLU gives output 0 and zero local gradient here.
- Leaky ReLU retains a nonzero negative-side path.
- It does not guarantee that every earlier gradient is large or that predictions improve.

Meaning

Activation choice changes how an observed output error can influence earlier weights; its practical effect also depends on the rest of the network.

Period 1 checkpoint: explain the representation

1. Why can more optimization steps not solve XOR with one affine threshold?
2. For $h_1 = \mathrm{ReLU}(x_1 + x_2)$ and $h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$, calculate $h_1 - 2h_2$ at $(1, 1)$.
3. A dense network $5 \to 4 \to 2$ has how many parameters?

Answers

$(2) - 2(1) = 0$; the parameter count is $(5 + 1)4 + (4 + 1)2 = 34$. XOR needs a changed representation, not simply repeated updates.

Carry forward

Architecture tells us what can be represented. Next we calculate one prediction and measure its error.

Without a nonlinear activation, depth collapses

Two affine layers without an activation give

$$h = W_1 x + b_1,$$
$$z = W_2 h + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2).$$

- The composition is still one affine map of x.
- Adding linear layers changes the parameterization, not this basic function class.
- A nonlinear hidden activation prevents this collapse.

Scalar example

$h = 2x + 1$, $z = 3h - 4$ gives $z = 6x - 1$: one straight line.

Connection to XOR

A final sigmoid alone still places the 0.5 boundary at one affine score equal to zero.

Compose two affine layers with actual matrices

$$W_1 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 2 & 1 \end{bmatrix}, \quad b_1 = (1,0)^\top, \quad b_2 = -1.$$

$$W_2 W_1 = (3,1), \qquad W_2 b_1 + b_2 = 1, \qquad f(x) = 3x_1 + x_2 + 1.$$

Check at $x = (1,2)$

The hidden affine values are $(4,-1)$, so the second layer gives $2(4) - 1 - 1 = 6$. The collapsed expression also gives 6.

Meaning

Different parameter matrices can describe one affine map. Without a hidden nonlinearity, the extra layer does not create a bent decision boundary.

A hidden ReLU prevents that affine collapse

Use the previous matrices, but activate the hidden values:

$$f(x) = 2\,\mathrm{ReLU}(x_1 + x_2 + 1) + \mathrm{ReLU}(x_1 - x_2) - 1.$$

Input	Pre-activation	Activated values	Output
$(1, 2)$	$(4, -1)$	$(4, 0)$	7
$(-2, 0)$	$(-1, -2)$	$(0, 0)$	-1

- Different input regions activate different subsets of units.
- The resulting expression is piecewise affine, rather than one affine map.

Meaning

The nonlinear step changes the function class even though the surrounding matrix operations are unchanged.

Depth composes reusable features

$$f(x) = f_L\big(f_{L-1}(\cdots f_1(x))\big).$$

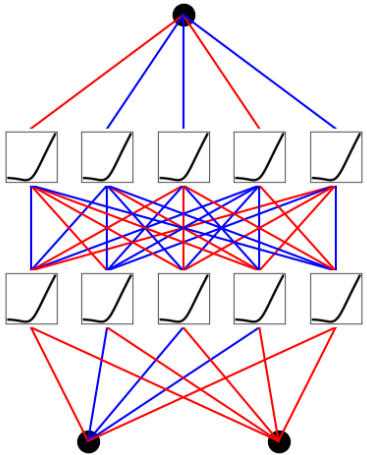
Idea	Benefit	Boundary
Composition	reuse intermediate computations	useful structure must fit the task
Width	more features at one stage	parameters and data needs increase
Depth	build features from earlier features	optimization can become difficult

- Universal approximation describes existence with sufficient capacity.
- It does not promise a small model, successful training, or good generalization.

Image intuition

Edges can combine into parts; parts can combine into object evidence. Lecture 16 makes the spatial connections explicit.

Depth composes reusable features



Reuse a useful intermediate feature in several outputs

Let $h_1(x)$ measure one pattern and $h_2(x)$ another. Two task scores can use the same representation:

$$z_A = 2h_1 - h_2 + 0.1, \qquad z_B = -h_1 + 3h_2 - 0.2.$$

For $h = (0.4, 0.7)$, $z_A = 0.2$ and $z_B = 1.5$.

- The feature computation is shared instead of duplicated for each score.
- Different outputs can place different importance on the same features.
- Training sends contributions from both output paths back to the shared layers.

Meaning

Depth and shared representations are useful when the target contains reusable structure; this is a reason to choose architecture, not a promise that every deeper model wins.

Batch computation: keep one convention throughout

For B examples stored as rows:

$$Z_1 = XW_1^\top + \mathbf{1}b_1^\top, \quad H = \phi(Z_1), \quad Z_2 = HW_2^\top + \mathbf{1}b_2^\top.$$

Object	Shape for $d = 4, m = 3, K = 2$	Role
X	$B \times 4$	one row per example
W_1	3×4	same weights as the single-example formula
Z_1, H	$B \times 3$	one row of hidden values per example
W_2	2×3	maps hidden coordinates to output scores
Z_2	$B \times 2$	one row per prediction

Bias broadcasting

The same bias vector is added to every row; it is not a separate parameter for each example.

Our running network: all parameters are given

For one example, use column vectors and ReLU hidden units:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad y = 1, \quad W_1 = \begin{bmatrix} 0.1 & 0.2 \\ -0.3 & 0.4 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

$$z_1 = W_1 x + b_1, \quad h = \mathrm{ReLU}(z_1), \quad v = \begin{bmatrix} 0.5 \\ -0.2 \end{bmatrix}, \quad c = 0.1, \quad s = v^\top h + c.$$

- W_1, b_1 produce two hidden features; v, c produce one binary-class score.
- We will compute h, s, $p = \sigma(s)$, then binary cross-entropy.
- Keep these values: the backward pass needs the same intermediates.

Forward pass: compute every hidden value

$$
\begin{aligned}
z_{1,1} &= 0.1(1) + 0.2(2) = 0.5, \\
z_{1,2} &= -0.3(1) + 0.4(2) = 0.5, \\
h &= (0.5, 0.5)^\top, \\
s &= 0.5(0.5) - 0.2(0.5) + 0.1 = 0.25, \\
p &= \sigma(0.25) \approx 0.562177.
\end{aligned}
$$

Interpret the result

The model assigns about 56.2% probability to class 1 for this input.

- The hidden values are features, not probabilities.
- The output score s is a logit; sigmoid changes it into a probability.
- A threshold produces a class label, but training will use the probability and target.

A hidden change matters through its output connection

Keep $v = (0.5, -0.2)$ and $c = 0.1$. Starting from $h = (0.5, 0.5)$:

Hidden change	New score	New positive probability
h_1 increases by 0.1	0.30	0.574443
h_2 increases by 0.1	0.23	0.557248

- The first positive output weight raises the score.
- The second negative output weight lowers it.
- More hidden activation does not always mean more positive-class evidence.

Meaning

Read an activation together with the connections that use it. This controlled intermediate change illustrates forward influence before we derive the corresponding gradients.

A batch can contain active and inactive hidden branches

Use the running network for $x^{(1)} = (1, 2)$ and $x^{(2)} = (2, 1)$:

$$Z_1 = \begin{bmatrix} 0.5 & 0.5 \\ 0.4 & -0.2 \end{bmatrix}, \qquad H = \begin{bmatrix} 0.5 & 0.5 \\ 0.4 & 0 \end{bmatrix}.$$

$$s = (0.25, 0.30)^\top, \qquad p \approx (0.562177, 0.574443)^\top.$$

- Each row uses identical model parameters.
- The second ReLU is active for the first example and inactive for the second.
- The different probabilities arise from the different inputs, not different output weights.

Meaning

A zero response for one example does not mean a hidden unit is inactive on the entire dataset.

Binary cross-entropy rewards the true class

$$L = -y \log p - (1 - y) \log(1 - p).$$

For our example $y = 1$, $p = 0.562177$:

$$L = -\log(0.562177) \approx 0.575939.$$

p when $y = 1$	Loss	Meaning
0.9	0.105	high probability on the true class
0.5	0.693	uncertain prediction
0.1	2.303	confidently favors the wrong class

Likelihood connection

Minimizing this loss maximizes the Bernoulli likelihood of the observed label.

Labels are not probabilities

The target y is observed; p is the model's prediction.

The dataset likelihood multiplies probabilities of observed labels

For targets $(1, 0)$ and predicted positive probabilities $(0.562177, 0.574443)$:

$$\mathcal{L} = p_1(1 - p_2) \approx 0.23924.$$

$$-\log \mathcal{L} = -\log p_1 - \log(1 - p_2) \approx 0.575939 + 0.854355 = 1.430294.$$

- Each factor is the probability assigned to its observed label.
- Logarithms turn multiplication across examples into addition of losses.
- Mean loss is 0.715147 for this two-example batch.

Meaning

The second example can be wrong even while the first is correctly classified. Training combines their evidence rather than judging only one thresholded prediction.

Regression needs a numeric output, not a class probability

For a real-valued target:

$$\hat{y} = v^\top h + c, \qquad L = \tfrac{1}{2}(\hat{y} - y)^2, \qquad \frac{\partial L}{\partial \hat{y}} = \hat{y} - y.$$

Target	Prediction	Loss	Output derivative
$y = 2$	$\hat{y} = 1.5$	0.125	-0.5
$y = 2$	$\hat{y} = 3$	0.5	1

- A linear output can predict values outside $[0, 1]$.
- The half-squared loss is convenient for differentiation.
- MSE corresponds to a Gaussian observation model with a suitable variance assumption.

Same hidden network, different task

Changing the prediction target changes the output and the loss.

Squared and absolute error react differently to an outlier

For predictions $(0, 0, 0)$ and targets $(1, 1, 10)$:

$$\text{mean half-squared loss} = (0.5 + 0.5 + 50)/3 = 17,$$

$$\text{mean absolute loss} = (1 + 1 + 10)/3 = 4.$$

- Half-squared-error output derivatives are $(-1, -1, -10)$ before averaging.
- Absolute-error subgradients are $(-1, -1, -1)$ here.
- Squared error gives the large residual much greater influence.

Meaning

The loss expresses which errors receive the strongest training pressure. Choose it for the target and error cost; changing the loss also changes the gradients.

Softmax compares all class scores together

For K mutually exclusive classes:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad L = -\log p_y.$$

Three-class example

$z = (2, 1, 0)$ gives $p \approx (0.665, 0.245, 0.090)$. If the target is class 1, $L \approx 0.408$.

- The probabilities sum to one; increasing one score affects all probabilities.
- Subtracting the same constant from every score leaves softmax unchanged.
- Multi-label tasks have several independent yes/no targets and commonly use sigmoid outputs instead.

Output size

A K-class single-label task usually has K logits; the target specifies one class index.

Softmax depends on relative scores, not a common offset

$$z = (2, 1, 0), \qquad z' = z + (100, 100, 100) = (102, 101, 100).$$
$$\mathrm{softmax}(z) = \mathrm{softmax}(z') \approx (0.66524, 0.24473, 0.09003).$$

- Multiplying all exponentials by e^{100} cancels between numerator and denominator.
- The score differences $z_1 - z_2$ and $z_2 - z_3$ remain one.
- Subtracting the maximum preserves probabilities and improves numerical stability.

Class comparison

$p_1/p_2 = e^{z_1 - z_2} = e$. Relative logits determine the relative odds of the two classes.

Multi-label outputs answer separate yes/no questions

Suppose an image contains a cat and a car, but no person. The target is $y = (1, 1, 0)$.

$$p = (0.8, 0.7, 0.1), \qquad L = -\log 0.8 - \log 0.7 - \log 0.9 \approx 0.6852.$$

- These three sigmoid probabilities need not sum to one.
- Each label contributes a binary loss; averaging over labels is a separate reduction choice.
- A softmax target selecting only one of the three would describe a different task.

Meaning

Read the label semantics before choosing the output activation. Several objects can coexist even when the model has only three named categories.

Compute classification losses from logits

Binary case

$$L(s, y) = \log(1 + e^s) - ys, \qquad \frac{\partial L}{\partial s} = \sigma(s) - y.$$

Use a stable softplus implementation rather than evaluating e^s naively.

Multiclass case

$$L(z, y) = -z_y + \operatorname{logsumexp}(z),$$

$$\operatorname{logsumexp}(z) = m + \log \sum_j e^{z_j - m}, \qquad m = \max_j z_j.$$

- Large logits can overflow a naive exponential.
- Standard combined loss functions apply these stable identities.
- Supply raw logits when the API expects logits; do not apply softmax twice.

A stable binary loss handles a large logit

For $s = 50$:

$$L(s, y) = \max(s, 0) - sy + \log(1 + e^{-|s|}).$$

Target	Loss approximately	Interpretation
1	$e^{-50} \approx 1.93 \times 10^{-22}$	confidently supports the true class
0	50	confidently supports the wrong class

- The same score can be excellent or terrible depending on the observed target.
- Use `log1p(exp(-abs(s)))` for the final term, or a tested BCE-with-logits implementation. This avoids exponential overflow and loss of the tiny correction.

Meaning

Numerically stable implementation preserves the intended statistical objective; it does not change which predictions the loss prefers.

Log-sum-exp stays finite after subtracting the maximum

Let $z = (1000, 1001, 999)$ and $m = 1001$:

$$\log \sum_j e^{z_j} = 1001 + \log(e^{-1} + 1 + e^{-2}) \approx 1001.407606.$$

For target class two:

$$L = -1001 + 1001.407606 = 0.407606.$$

- Every shifted exponential is at most one.
- The large common offset cancels from the final cross-entropy.
- The resulting probabilities are approximately $(0.24473, 0.66524, 0.09003)$.

Meaning

A large absolute logit does not require an enormous loss. The observed class score is compared with all competing class scores.

A batch loss combines example losses and regularization

$$J(\theta) = \frac{1}{B}\sum_{i=1}^{B} L_i(\theta) + \frac{\lambda}{2}\|W\|_F^2.$$

- The mean gives each batch a comparable loss scale.
- The data gradient is the mean of the individual example gradients.
- The penalty adds λW to the selected weight gradients.
- Biases are often excluded from this penalty; state the implementation choice.

A scale calculation

Losses $0.4, 0.8$ average to 0.6. Repeating both examples leaves the mean unchanged, but doubles a sum loss.

Keep the reduction consistent

Switching from mean to sum changes gradient magnitudes and the effective learning-rate scale.

Regularization adds its own gradient to a weight

For one weight $w = 0.4$, data gradient $g_{\text{data}} = -0.3$, and $\lambda = 0.1$:

$$J = L + \tfrac{\lambda}{2} w^2, \quad g_J = -0.3 + 0.1(0.4) = -0.26.$$

With $\eta = 0.1$:

$$w_{\text{new}} = 0.4 - 0.1(-0.26) = 0.426.$$

- Data asks the weight to increase; the penalty pulls it toward zero.
- The combined gradient gives a smaller increase than the data gradient alone (0.43).
- The penalty is added once under the stated mean-loss convention.

Meaning

Regularization changes the optimization objective, so its influence must be included in the update rather than treated as a separate afterthought.

A computation graph records dependencies

For a scalar unit, write the operations separately:

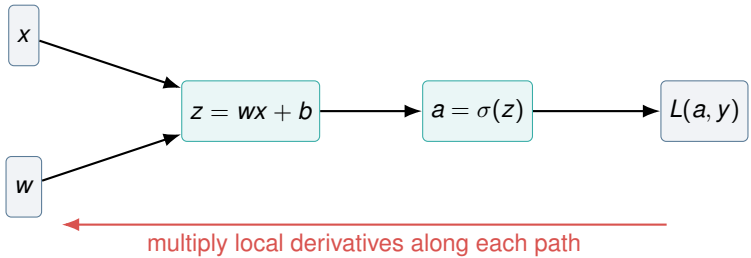
$$u = wx, \quad z = u + b, \quad a = \phi(z), \quad \hat{y} = va + c, \quad L = \tfrac{1}{2}(\hat{y} - y)^2.$$

Operation	Saved forward value	Local derivative
multiply $u = wx$	w, x	$\partial u/\partial w = x$
add $z = u + b$	none needed	$\partial z/\partial u = 1$
activate $a = \phi(z)$	z	$\partial a/\partial z = \phi'(z)$
predict $\hat{y} = va + c$	v, a	$\partial \hat{y}/\partial a = v$

Purpose

The backward calculation combines local derivatives along the dependency paths.

A computation graph records dependencies



Chain rule multiplies local effects

$$u = 2x, \qquad v = u^2, \qquad L = 3v.$$

At $x = 1$: $u = 2$, $v = 4$, $L = 12$.

$$\frac{dL}{dx} = \frac{dL}{dv}\frac{dv}{du}\frac{du}{dx} = 3 \cdot (2u) \cdot 2 = 24.$$

Small-change interpretation

Increasing x by 0.001 changes L by about $24(0.001) = 0.024$. The exact change is 0.024012.

- Each local derivative is evaluated at the saved forward value.
- The derivative measures a local change, not the total loss value.
- Backpropagation applies this rule efficiently to many connected operations.

Evaluate the local derivative at the forward value

For $a = \tanh(wx)$, let $w = 0.5$, $x = 2$:

$$z = wx = 1, \quad a \approx 0.761594, \quad \frac{\partial a}{\partial w} = (1 - a^2)x \approx 0.839949.$$

- Multiplication contributes input $x = 2$.
- Tanh contributes its slope at $z = 1$, not its slope at zero.
- Changing the input changes both the multiplication factor and the activation's evaluation point.

At $x = 0$

The weight derivative becomes zero because wx cannot change when the input is zero. This follows from the dependency, not from a saturated tanh.

When paths split, their gradient contributions add

Let $a = x^2$, $b = 3x$, and $L = a + b$.

$$\frac{dL}{dx} = \frac{\partial L}{\partial a}\frac{da}{dx} + \frac{\partial L}{\partial b}\frac{db}{dx} = 2x + 3.$$

At $x = 2$, the two paths contribute 4 and 3, so $dL/dx = 7$.

- The same input can influence the loss through several downstream units.
- Backward contributions are accumulated at a shared value.
- Reused parameters also accumulate contributions from every use.

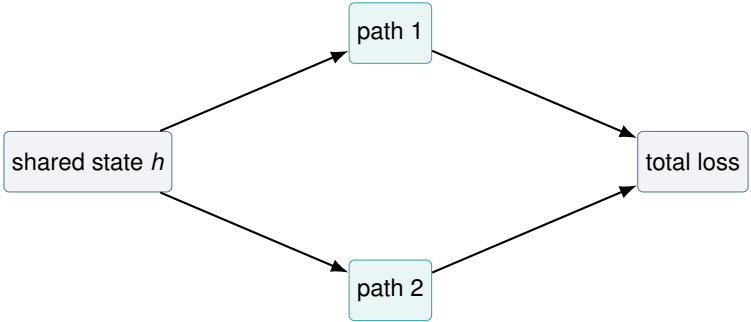
Next lecture

A convolution reuses one filter at many positions; its weight gradient sums all those contributions.

Do not discard a branch

Using only one outgoing path produces an incomplete gradient.

When paths split, their gradient contributions add



$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial h}\bigg|_{\text{path 1}} + \frac{\partial L}{\partial h}\bigg|_{\text{path 2}}$$

Repeated uses of the same parameter accumulate gradients

Let one parameter w appear twice:

$$a = wx_1, \quad b = wx_2, \quad L = (a+b)^2, \quad x_1 = 1, x_2 = 2, w = 0.5.$$

Then $a + b = 1.5$ and

$$\frac{\partial L}{\partial w} = 2(a+b)x_1 + 2(a+b)x_2 = 3 + 6 = 9.$$

- Each use of w contributes through its own dependency path.
- Using only the first multiplication would incorrectly return 3.
- Sharing ties parameter values, not their contributions to the objective.

Next lecture

A convolution filter repeats at many image positions, so exactly this accumulation principle applies there.

Forward values are also backward inputs

Layer or operation	Forward cache	Backward use
Dense layer	input activations	weight gradient
ReLU	pre-activation sign	zero or one derivative
Sigmoid	output probability	$p(1-p)$ when needed
Binary cross-entropy with logit	logit or probability	$p-y$

- Automatic differentiation records the operations needed to obtain derivatives.
- It does not choose the correct target, data split, or loss for us.
- A detached value can retain the right number but lose its dependency on earlier parameters.

Running example

We will reuse $x = (1, 2)$, $z_1 = h = (0.5, 0.5)$, and $p = 0.562177$ during backpropagation.

A minimal forward pass mirrors the equations

```
import torch
import torch.nn.functional as F
X = torch.tensor([[1., 2.]])       # B x 2
W1 = torch.tensor([[.1, .2], [-.3, .4]])
b1 = torch.zeros(2)
v = torch.tensor([[.5, -.2]])
c = torch.tensor([.1])
h = F.relu(X @ W1.T + b1)          # B x 2
logits = h @ v.T + c               # B x 1
target = torch.ones_like(logits)
loss = F.binary_cross_entropy_with_logits(logits, target)
```

Compare with hand calculation

$h = (0.5, 0.5)$, logit 0.25, loss about 0.57594.

Shape detail

The target matches the logit shape; do not let unintended broadcasting create extra examples.

Period 2 checkpoint: target, score, probability, loss

1. Our hidden activations are $(0.5, 0.5)$ and the logit is 0.25. Which quantity is a probability?
2. If the target changes from 1 to 0, does the same prediction become better or worse?
3. Why does a regression output usually not need sigmoid?
4. For two paths from x to the loss, do their derivative contributions add or multiply?

Answers

$p = \sigma(0.25) \approx 0.5622$ is the probability. For target 0, the loss becomes $-\log(1 - p) \approx 0.8259$. Parallel path contributions add.

Next step

The current target is still 1. We now trace its loss backward and update the same parameters.

Start backward at the binary output score

With sigmoid and binary cross-entropy:

$$\delta_s = \frac{\partial L}{\partial s} = p - y = 0.562177 - 1 = -0.437823.$$

Since $s = v^\top h + c$:

$$\nabla_v L = \delta_s h = \begin{bmatrix} -0.218912 \\ -0.218912 \end{bmatrix}, \qquad \frac{\partial L}{\partial c} = \delta_s = -0.437823.$$

Interpret the sign

The score is too low for target 1; decreasing the loss requires raising the score locally.

Do not apply sigmoid twice

The simplification $p - y$ already includes both the loss derivative and the sigmoid derivative.

Binary cross-entropy and sigmoid simplify together

$$\frac{\partial L}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p} = \frac{p-y}{p(1-p)}, \qquad \frac{\partial p}{\partial s} = p(1-p).$$

$$\frac{\partial L}{\partial s} = p - y.$$

- The two factors cancel when used together.
- For $y = 1$ and $p = 0.562177$, the logit gradient is -0.437823.
- Multiplying by another sigmoid derivative would apply the activation slope twice.

Meaning

The shortcut formula has a derivation. It belongs to this output-loss combination and cannot automatically be reused for arbitrary losses.

Send the gradient to each hidden unit

$$\frac{\partial L}{\partial h} = v\delta_s = \begin{bmatrix} 0.5 \\ -0.2 \end{bmatrix} (-0.437823) = \begin{bmatrix} -0.218912 \\ 0.087565 \end{bmatrix}.$$

Because both $z_{1,j} = 0.5 > 0$, ReLU derivatives are 1:

$$\delta_1 = \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h} \odot \mathrm{ReLU}'(z_1) = \begin{bmatrix} -0.218912 \\ 0.087565 \end{bmatrix}.$$

- Increasing the first hidden value raises the score and helps this example.
- Increasing the second hidden value lowers the score because $v_2 < 0$.
- A negative ReLU pre-activation would set that local branch derivative to zero.

An inactive ReLU blocks only its local branch

For the second example $x = (2, 1)$, the second hidden pre-activation is -0.2.

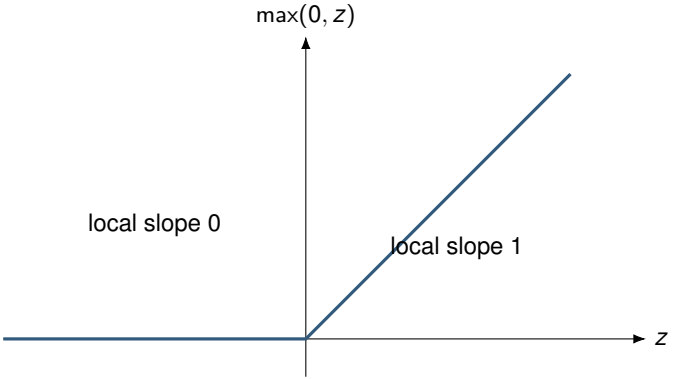
$$\mathrm{ReLU}'(-0.2) = 0, \qquad \frac{\partial L}{\partial z_{1,2}} = \frac{\partial L}{\partial h_2} \cdot 0 = 0.$$

- Its incoming weights get zero gradient from this example through this path.
- The output bias and the active first hidden unit can still receive gradients.
- Other examples may put the same hidden unit in the positive region.

Meaning

Do not confuse a zero derivative on one example with a permanently inactive unit or a completely non-learning network.

An inactive ReLU blocks only its local branch



Each first-layer weight gets input times error

For $z_1 = W_1 x + b_1$:

$$\nabla_{W_1} L = \delta_1 x^\top = \begin{bmatrix} -0.218912 \\ 0.087565 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} -0.218912 & -0.437823 \\ 0.087565 & 0.175129 \end{bmatrix}.$$

$$\nabla_{b_1} L = \delta_1 = (-0.218912,\ 0.087565)^\top.$$

- Row j collects the gradients entering hidden unit j.
- Column k uses input feature x_k.
- A larger input can produce a larger weight gradient for the same hidden error.

Shape check

$(2 \times 1)(1 \times 2) = 2 \times 2$, exactly the shape of W_1.

The loss also has a gradient with respect to the input

Using the running-example hidden error $\delta_1 = (-0.218912, 0.087565)^\top$:

$$\frac{\partial L}{\partial x} = W_1^\top \delta_1 \approx \begin{bmatrix} -0.048161 \\ -0.008756 \end{bmatrix}.$$

- The first coordinate gets $0.1(-0.218912) + (-0.3)(0.087565)$.
- The second gets $0.2(-0.218912) + 0.4(0.087565)$.
- These describe local loss sensitivity to input perturbations at this point.

Interpret carefully

An input gradient is not a causal effect or a complete explanation of the prediction. It depends on the current model, input, and chosen loss.

One gradient step lowers the running-example loss

With learning rate $\eta = 0.1$, update all parameters simultaneously:

$$\theta_{\text{new}} = \theta - \eta \nabla_\theta L.$$

$$W_{1,\text{new}} \approx \begin{bmatrix} 0.121891 & 0.243782 \\ -0.308756 & 0.382487 \end{bmatrix}, \quad b_{1,\text{new}} \approx \begin{bmatrix} 0.021891 \\ -0.008756 \end{bmatrix},$$

$$v_{\text{new}} \approx (0.521891, -0.178109)^\top, \quad c_{\text{new}} \approx 0.143782.$$

Forward result	Before	After
h	$(0.5, 0.5)$	$(0.631347, 0.447461)$
s	0.25	0.393580
p	0.562177	0.597144
L	0.575939	0.515597

A gradient direction can be useful while a step is too large

Consider $L(w) = \frac{1}{2}w^2$, $w = 1$, so $g = 1$:

Learning rate	New w	New loss	Result
0.1	0.9	0.405	decrease
1	0	0	minimum
3	-2	2	increase

- All three steps initially move in the negative-gradient direction.
- A local derivative does not guarantee improvement for an arbitrarily large step.
- Recompute the objective after updating rather than relying on the sign alone.

Meaning

The optimizer uses both gradient direction and step size. Correct derivatives do not remove the need to control the update.

Multiple output paths add at a hidden unit

For hidden vector h and output scores $z_2 = W_2 h + b_2$:

$$\frac{\partial L}{\partial h_j} = \sum_k W_{2,kj}\, \delta_{2,k}, \qquad \delta_1 = (W_2^\top \delta_2) \odot \phi'(z_1).$$

Two outputs

If one hidden value has outgoing weights $0.4, -0.2$ and the two score gradients are $0.3, -0.5$, its incoming gradient is $0.4(0.3) + (-0.2)(-0.5) = 0.22$.

- Matrix multiplication performs the sum over downstream paths.
- The activation derivative is applied after those contributions combine.
- For softmax with cross-entropy, $\delta_2 = p - \mathrm{onehot}(y)$.

Softmax cross-entropy produces a signed error vector

For $p = (0.66524, 0.24473, 0.09003)$ and target class two:

$$\delta = p - (0, 1, 0) = \big(0.66524, -0.75527, 0.09003\big).$$

- The true-class score receives a negative derivative, encouraging an increase.
- Other scores receive positive derivatives, encouraging decreases locally.
- The components sum to zero, consistent with invariance to a common logit offset.

Weight gradient

For hidden feature $h_j = 0.4$, multiply each score error by that feature value:

$$h_j \delta \approx (0.26610, -0.30211, 0.03601)^\top.$$

Meaning

One loss coordinates all competing scores; the target selects which class should gain relative support.

Batch backpropagation has predictable shapes

For row-wise examples, $Z = AW^\top + \mathbf{1}b^\top$, with $A \in \mathbb{R}^{B\times d}$ and $W \in \mathbb{R}^{m\times d}$:

$$G = \frac{\partial J}{\partial Z} \in \mathbb{R}^{B\times m}, \quad \nabla_W J = G^\top A, \quad \nabla_b J = G^\top \mathbf{1}, \quad \frac{\partial J}{\partial A} = GW.$$

Gradient	Shape	What is combined
$G^\top A$	$m \times d$	contributions from all examples
$G^\top \mathbf{1}$	$m \times 1$	one bias contribution per example
GW	$B \times d$	all output paths for each input

Mean convention

For mean loss, G already includes $1/B$. Do not divide the weight gradient by B a second time.

Average the gradients of the two running examples

For targets $(1, 0)$, the output-logit errors are $(-0.437823, 0.574443)$.

$$\nabla_v J = \tfrac{1}{2}\left[(-0.437823)(0.5, 0.5) + (0.574443)(0.4, 0)\right] \approx (0.005433, -0.109456).$$

$$\partial J/\partial c = (-0.437823 + 0.574443)/2 \approx 0.068310.$$

- The two examples ask the first output weight to change in opposite directions.
- Their mean can be much smaller than either individual contribution.
- The second example gives no contribution through its inactive second hidden feature.

Meaning

A batch update compromises among examples. Apply the mean factor once and keep the same parameters for every contribution.

A complete training step follows one order

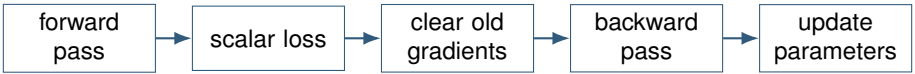
```
optimizer.zero_grad()
hidden = F.relu(X @ W1.T + b1)
logits = hidden @ v.T + c
loss = F.binary_cross_entropy_with_logits(logits, target)
loss.backward()
optimizer.step()
```

- Parameters must be registered with the optimizer and require gradients.
- Clear old gradients before computing this batch's gradients.
- Compute all backward contributions before changing parameters.
- The optimizer uses gradients; backpropagation computes them.

Distinct roles

The architecture defines the function, the loss defines the objective, backpropagation computes derivatives, and the optimizer changes parameters.

A complete training step follows one order



Automatic differentiation should reproduce the hand calculation

```
X = torch.tensor([[1., 2.]])
W1 = torch.tensor([[.1,.2],[-.3,.4]], requires_grad=True)
b1 = torch.zeros(2, requires_grad=True)
v = torch.tensor([[.5,-.2]], requires_grad=True)
c = torch.tensor([.1], requires_grad=True)
logit = F.relu(X @ W1.T + b1) @ v.T + c
F.binary_cross_entropy_with_logits(logit,
    torch.ones_like(logit)).backward()
```

Expected gradient

W_1 should receive approximately $[[-0.218912, -0.437823], [0.087565, 0.175129]]$.

Meaning

A library result is most informative when we can compare it with a known mathematical result on a small deterministic case.

Check one derivative by a small perturbation

For parameter θ_j:

$$g_j^{\text{num}} = \frac{L(\theta + \epsilon e_j) - L(\theta - \epsilon e_j)}{2\epsilon}.$$

For our original $v_1 = 0.5$, $\epsilon = 10^{-4}$:

$$L(v_1 + \epsilon) \approx 0.575917530, \quad L(v_1 - \epsilon) \approx 0.575961312, \quad g^{\text{num}} \approx -0.218912.$$

$$g^{\text{backprop}} = (p - y)h_1 \approx -0.218912.$$

- Change only the selected parameter and hold all others fixed.
- Use a tiny deterministic model and adequate numerical precision.
- Avoid evaluating at ReLU kinks or using random layers without fixing their state.

Perturbation size controls a numerical derivative check

For $f(w) = w^3$ at $w = 2$, the exact derivative is 12:

$$\frac{f(2+\epsilon) - f(2-\epsilon)}{2\epsilon} = 12 + \epsilon^2.$$

ϵ	Central difference	Approximation error
0.1	12.01	0.01
0.01	12.0001	0.0001
0.001	12.000001	0.000001

- Decreasing the step reduces this truncation error.
- In floating-point arithmetic, excessively small differences can suffer cancellation.

Meaning

Use several sensible perturbations and adequate precision; a single tiny number is not automatically the most trustworthy check.

Identical hidden units stay identical

Suppose two hidden units start with identical incoming parameters and identical outgoing roles.

- They receive the same input and compute the same activation.
- They receive the same backward signal.
- Their parameter updates are identical, so they fail to learn different features.

Why not all zeros?

Zero initialization in a symmetric hidden network keeps units indistinguishable. It can also block early gradients through zero outgoing weights.

Practical choice

Use different random initial weights with a controlled scale. Biases can often start at zero.

Not every zero is forbidden

A zero bias is not the same problem as setting every hidden weight to the same value.

Identical input weights alone are not the full symmetry condition

Two hidden units may share the same input activation h, but receive different output weights v_1, v_2:

$$\delta_1 = v_1 \delta_{\text{out}} \phi'(z), \qquad \delta_2 = v_2 \delta_{\text{out}} \phi'(z).$$

Numbers

For $v_1 = 1$, $v_2 = -1$, $\delta_{\text{out}} = 0.2$, and activation slope 1, the hidden errors are 0.2 and -0.2.

- Equal incoming values can still get different updates if outgoing roles differ.
- The all-identical initialization problem includes the symmetric outgoing structure.

Meaning

Explain the actual symmetry rather than treating every pair of equal values as permanently indistinguishable.

Initialization scale uses the number of inputs

Under approximate independence and zero-mean assumptions:

$$\mathrm{Var}(z) \approx d\ \mathrm{Var}(w)\ \mathrm{Var}(x).$$

Common choice	Weight variance	Main use	Key intuition
Xavier/Glorot	$2/(d+m)$	tanh or similar layers	balance forward/backward scales
He	$2/d$	ReLU hidden layers	compensate for discarded negative values

ReLU layer with $d = 100$

He variance is 0.02 and standard deviation is $\sqrt{0.02} \approx 0.1414$.

- Draw independent weights with the chosen variance.
- Variance and standard deviation are different quantities.
- These are principled starting rules, not guarantees for every architecture.

Read fan-in and fan-out from the layer you initialize

For $W \in \mathbb{R}^{m \times d}$, fan-in is d and fan-out is m.

Layer	He standard deviation	Glorot standard deviation
$16 \to 64$	$\sqrt{2/16} \approx 0.3536$	$\sqrt{2/80} \approx 0.1581$
$64 \to 16$	$\sqrt{2/64} \approx 0.1768$	$\sqrt{2/80} \approx 0.1581$

- He's forward-scale rule changes when the number of incoming signals changes.
- Glorot uses both counts under its balancing assumptions.

Meaning

Use the initialization rule appropriate to the activation and stated convention; do not reverse matrix dimensions or confuse variance with standard deviation.

Check the scale through a ReLU layer

Let the input second moment be $\mathbb{E}[x_j^2] \approx 1$, with $d = 100$ and He variance $2/d = 0.02$.

$$\mathbb{E}[z^2] \approx 100(0.02)(1) = 2.$$

For a symmetric zero-mean pre-activation distribution:

$$\mathbb{E}[\mathrm{ReLU}(z)^2] \approx \tfrac{1}{2}\mathbb{E}[z^2] \approx 1.$$

Why the factor two?

The pre-activation scale grows to about two so the ReLU output second moment stays near one.

Precise quantity

This uses the second moment. ReLU outputs have a positive mean, so their variance is not generally equal to their second moment.

Contrast

Using variance 0.0001 gives pre-activation second moment about 0.01, which can shrink signals repeatedly.

ReLU second moment and variance are different

If $z \sim N(0,1)$ and $a = \mathrm{ReLU}(z)$:

$$\mathbb{E}[a] = 1/\sqrt{2\pi} \approx 0.3989, \quad \mathbb{E}[a^2] = 1/2,$$

$$\mathrm{Var}(a) = \mathbb{E}[a^2] - (\mathbb{E}[a])^2 = 1/2 - 1/(2\pi) \approx 0.3408.$$

- The positive output mean explains why variance is smaller than the second moment.
- "Half of the squared signal remains" refers to the second moment for a symmetric input distribution.

Meaning

Name the quantity being controlled. Initialization arguments use approximate distribution assumptions; their wording should not silently replace second moment by variance.

Products of local factors can shrink or grow

For a hidden layer:

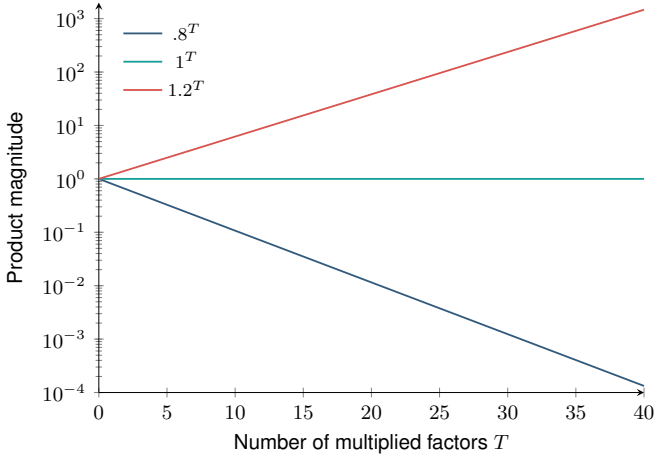
$$\delta_l = (W_{l+1}^\top \delta_{l+1}) \odot \phi'(z_l).$$

Observed behavior	Possible mechanism	Useful response
Early layers barely change	small products of derivatives	check scale and activation saturation
Loss or gradients become non-finite	large products or unstable arithmetic	check loss, learning rate, gradient scale
Some ReLU units never activate	negative pre-activations for all inputs	inspect inputs, initialization, learning rate

Connection

Initialization controls a starting scale; residual paths and normalization in later lectures provide additional ways to improve training.

Products of local factors can shrink or grow



Repeated local factors make depth sensitive to scale

Along a simple scalar path with eight equal derivative factors:

$$0.5^8 = 0.00390625, \qquad 1^8 = 1, \qquad 2^8 = 256.$$

- Small factors can greatly attenuate the backward signal.
- Large factors can greatly amplify it.
- Real networks also sum paths and multiply matrices, so this is an intuition example, not a universal scalar formula.

Observation to mechanism

If early layers barely update, inspect both activation slopes and weight scales. If gradients become non-finite, inspect the forward loss calculation and update size too.

Meaning

Depth makes scale choices important; no single activation or shortcut guarantees every gradient is well behaved.

Test learning first on a tiny deterministic dataset

1. Run the hand-calculated example and compare forward values and gradients.
2. Fit a small fixed subset with a model that has enough capacity.
3. Confirm the optimizer changes parameters and the chosen loss decreases.
4. Then inspect validation performance on data not used for fitting.

Finding	Interpretation	Next question
Cannot fit a few clean examples	implementation or optimization issue likely	are labels, gradients, and updates correct?
Fits training, fails validation	generalization or distribution issue	is capacity or data mismatch the cause?

Tiny fit is limited evidence

It shows the training path can work; it does not show the model generalizes.

Separate fitting failure from generalization failure with numbers

Model run	Training loss	Validation loss	First interpretation
A	0.90	0.95	poor fit on both sets
B	0.05	0.80	fit is strong; generalization is weak
C	0.20	0.23	smaller gap; compare task errors

- Run A may involve unsuitable capacity, optimization, labels, or input handling.
- Run B calls for examining capacity, data differences, and regularization.
- The table is not enough to prove one cause; inspect predictions and experimental conditions.

Meaning

Use observations to narrow a question. Do not make all poor outcomes mean overfitting or assume a low training loss is sufficient.

Compare the roles before choosing a component

Component	Question it answers	Running example
Hidden layers	What features can be represented?	two ReLU values
Output and loss	What is predicted and penalized?	Bernoulli probability and BCE
Backpropagation	How does each parameter affect loss?	local gradient $\partial L/\partial\theta$
Optimizer	How are parameters changed?	one SGD step with $\eta = 0.1$
Validation	Does fitting help new examples?	requires held-out data

Concrete diagnosis

Changing the optimizer cannot make a linear threshold represent XOR; adding layers cannot fix wrong target labels.

The complete story: features, prediction, error, update

1. A hidden layer changes the representation; nonlinear activations prevent affine collapse.
2. Forward propagation computes activations, then task-specific scores.
3. The target and likelihood determine the output transformation and loss.
4. Backpropagation multiplies local effects along paths and sums shared contributions.
5. Weight gradients combine incoming activations with outgoing error signals.
6. A controlled initialization and a small numerical example make learning easier to understand.

Our calculation

$L : 0.575939 \rightarrow 0.515597$ after one simultaneous step. The model raised the probability of the true class from 0.562177 to 0.597144.

Next lecture

CNNs keep this learning process while imposing local connections and shared weights.

Exit exercise: trace one changed example

Keep the original network parameters, but use $x = (2, 1)^\top$ and $y = 0$.

1. Compute z_1, h, s, p, and the binary loss.
2. Find $\delta_s = p - y$, then the two hidden pre-activation gradients.
3. Explain why one hidden branch receives zero gradient.

Results

$z_1 = (0.4, -0.2)^\top$, $h = (0.4, 0)^\top$, $s = 0.3$, $p \approx 0.574443$, $L \approx 0.854355$. Then $\delta_1 \approx (0.287221, 0)^\top$.

Explain the direction

For target 0, gradient descent should locally lower the output score. The second ReLU is inactive on this example.

Finish the changed-example weight gradients

For $x = (2, 1)$, $y = 0$, $h = (0.4, 0)$, and $\delta_s = 0.574443$:

$$\nabla_v L \approx (0.229777, 0), \quad \partial L/\partial c = 0.574443,$$

$$\delta_1 \approx (0.287221, 0)^\top, \quad \nabla_{W_1} L \approx \begin{bmatrix} 0.574443 & 0.287221 \\ 0 & 0 \end{bmatrix}.$$

- The active first unit contributes to both input-weight gradients.
- The inactive second unit contributes zero through its ReLU branch.
- Positive output-score error asks gradient descent to reduce the positive-class score for this target.

Meaning

The same forward and backward rules work after changing the input and label. Explaining the zero row is as important as reproducing the decimals.